Notebook 11 - VQC auf IBM Hardware (ibm_kingston)

Dieses Notebook führt VQC-Klassifikation auf echter IBM Quantum Hardware durch. Verwendet werden die **optimierten Hyperparameter** aus der Hyperparameter-Analyse.

Datensatz	Optimizer	Iterationen	optimization_level
Iris	COBYLA	100	1
BC	COBYLA	50	1

Erfasste Metriken:

- Accuracy & F1 (Vergleich zu Simulator)
- Trainings- und Inferenzzeit
- Circuit-Tiefe nach Transpilierung (Effekt des Pass Managers)
- Job Queue-Zeit (praktische NISQ-Einschränkung)
- Fehlerrate aus Bitstring-Verteilung

Imports

In [1]:
```
import time
import csv
import os
import numpy as np

from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, f1_score, classification_report

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit_algorithms.optimizers import COBYLA

from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler
from qiskit.transpiler import generate_preset_pass_manager
```

In [2]:
```
pip install qiskit-algorithms --break-system-packages
```

```
Collecting qiskit-algorithms
  Downloading qiskit_algorithms-0.4.0-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: qiskit>=1.0 in /home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages (from qiskit-algorithms) (2.4.1)
Requirement already satisfied: scipy>=1.4 in /home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages (from qiskit-algorithms) (1.17.1)
Requirement already satisfied: numpy>=1.17 in /home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages (from qiskit-algorithms) (2.4.4)
Requirement already satisfied: rustworkx>=0.15.0 in /home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages (from qiskit>=1.0->qiskit-algorithms) (0.17.1)
Requirement already satisfied: dill>=0.3 in /home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages (from qiskit>=1.0->qiskit-algorithms) (0.4.1)
Requirement already satisfied: stevedore>=3.0.0 in /home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages (from qiskit>=1.0->qiskit-algorithms) (5.7.0)
Requirement already satisfied: typing-extensions in /home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages (from qiskit>=1.0->qiskit-algorithms) (4.15.0)
Downloading qiskit_algorithms-0.4.0-py3-none-any.whl (327 kB)
Installing collected packages: qiskit-algorithms
Successfully installed qiskit-algorithms-0.4.0
Note: you may need to restart the kernel to use updated packages.
```

Konfiguration

In [2]:
```python
OPTIMIZATION_LEVEL = 1      # Pass Manager Level - anpassbar für Vergleich
BACKEND_NAME       = 'ibm_kingston'
RANDOM_STATE       = 42
TEST_SIZE          = 0.3
ERGEBNISSE_CSV     = 'ergebnisse.csv'
HARDWARE_CSV       = 'ergebnisse_hardware.csv'  # separate Datei für Hardwar
```

Verbindung zu IBM Quantum

In [3]:
```python
service = QiskitRuntimeService()
backend = service.backend(BACKEND_NAME)
print(f"Backend: {backend.name}")
print(f"Qubits:  {backend.num_qubits}")
print(f"Status:  {backend.status().status_msg}")
print(f"Pending jobs in queue: {backend.status().pending_jobs}")
```

```
qiskit_runtime_service.__init__:WARNING:2026-05-01 12:15:19,577: Instance was not set at service instantiation. Free and trial plan instances will be prioritized. Based on the following filters: (tags: None, region: us-east, eu-de), and available plans: (open), the available account instances are: open-instance. If you need a specific instance set it explicitly either by using a saved account with a saved default instance or passing it in directly to QiskitRuntimeService().
qiskit_runtime_service.backends:WARNING:2026-05-01 12:15:19,580: Using instance: open-instance, plan: open
```

```
Backend: ibm_kingston
Qubits:  156
Status:  active
Pending jobs in queue: 0
```

Pass Manager konfigurieren

In [4]:
```python
# Hinweis: transpile() ruft intern generate_preset_pass_manager auf.
# Hier wird der Manager explizit erstellt, um optimization_level
# transparent zu dokumentieren und Circuit-Kennzahlen erfassen zu können.
# Quelle: IBM Quantum Documentation, "Transpiler stages"
pm = generate_preset_pass_manager(
    optimization_level=OPTIMIZATION_LEVEL,
    backend=backend
)
print(f"Pass Manager konfiguriert (optimization_level={OPTIMIZATION_LEVEL})"
```

```
Pass Manager konfiguriert (optimization_level=1)
```

Hilfsfunktionen

In [5]:
```python
def prepare_data(dataset_name: str):
    if dataset_name == 'iris':
        data  = load_iris()
        label = 'Iris (3 Klassen, 2 Features, fair)'
        f1_avg = 'weighted'
    else:
        data  = load_breast_cancer()
        label = 'Breast Cancer (2 Klassen, 2 Features, fair)'
        f1_avg = 'binary'

    X, y = data.data[:, :2], data.target
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
    )
    scaler = MinMaxScaler(feature_range=(0, np.pi))
    X_train_sc = scaler.fit_transform(X_train)
    X_test_sc  = scaler.transform(X_test)
    return X_train_sc, X_test_sc, y_train, y_test, label, f1_avg, data

def circuit_info(feature_map, ansatz, pm):
    qc = feature_map.compose(ansatz)
    qc_t = pm.run(qc)
    return {
        'depth':        qc_t.depth(),
        'gate_counts':  dict(qc_t.count_ops()),
        'num_qubits':   qc_t.num_qubits,
        'width':        qc_t.width(),        # Qubits + klassische Bits
        'size':         qc_t.size(),         # Gesamtzahl Operationen
        'num_2q_gates': sum(v for k, v in qc_t.count_ops().items()
                            if qc_t.num_qubits >= 2 and k in ['cx', 'ecr', '
    }

def circuit_info(feature_map, ansatz, pm):
```

```
    qc   = feature_map.compose(ansatz)   # logischer Circuit
    qc_t = pm.run(qc)                    # transpiliert
    return {
        'depth':        qc_t.depth(),
        'gate_counts':  dict(qc_t.count_ops()),
        'num_qubits':   qc.num_qubits,   # ← qc statt qc_t
        'width':        qc_t.width(),
        'size':         qc_t.size(),
        'num_2q_gates': sum(v for k, v in qc_t.count_ops().items()
                            if k in ['cx', 'ecr', 'cz']),
    }

def append_hardware_csv(row: dict):
    fieldnames = [
        'Modell','Datensatz','Backend','Optimizer','Iterationen',
        'Accuracy','F1','Trainingszeit_s','Inferenzzeit_s',
        'Queue_Zeit_s','Circuit_Depth','Gate_Counts', 'Num_2q_Gates',
        'optimization_level','Job_ID'
    ]
    file_exists = os.path.isfile(HARDWARE_CSV)
    with open(HARDWARE_CSV, 'a', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        if not file_exists:
            writer.writeheader()
        writer.writerow(row)
    print(f"  → Hardware-Details gespeichert in {HARDWARE_CSV}")
```

Lauf 1 - VQC Iris auf IBM Hardware

Konfiguration: COBYLA, 100 Iterationen, optimization_level=1

In [ ]:
```
X_train, X_test, y_train, y_test, ds_label, f1_avg, iris = prepare_data('iri

feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
ansatz      = RealAmplitudes(num_qubits=2, reps=2)
sampler     = Sampler(backend)

vqc_iris = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=COBYLA(maxiter=100),
    sampler=sampler,
    pass_manager=pm,
)

# --- Circuit-Eigenschaften vor dem Training ---
info = circuit_info(feature_map, ansatz, pm)
print(f"Circuit depth:      {info['depth']}")
print(f"Gate counts:        {info['gate_counts']}")
print(f"Anzahl Qubits:      {info['num_qubits']}")
print(f"Circuit size:       {info['size']}")
print(f"2-Qubit-Gates:      {info['num_2q_gates']}")

# --- Queue-Zeit messen ---
```

```python
print("\nStarte VQC Training (Iris, Hardware)...")
t_queue_start = time.time()
# Erster Circuit-Submit startet Queue-Wartezeit
# (Zeit bis Training startet wird als Queue-Zeit approximiert)

t_train_start = time.time()
vqc_iris.fit(X_train, y_train)
train_time = round(time.time() - t_train_start, 4)

t0 = time.time()
y_pred = vqc_iris.predict(X_test)
infer_time = round(time.time() - t0, 4)

acc = accuracy_score(y_test, y_pred)
f1  = f1_score(y_test, y_pred, average=f1_avg)

print(f"\nAccuracy: {acc:.4f}  |  F1: {f1:.4f}")
print(f"Training: {train_time}s  |  Inferenz: {infer_time}s")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# --- Job-Details ausgeben ---
# Hinweis: Bei VQC werden intern viele Jobs abgesetzt.
# Job-ID des letzten Jobs:
print(f"Backend: {backend.name}")
print(f"optimization_level: {OPTIMIZATION_LEVEL}")
print(f"Circuit depth: {info['depth']}")
print(f"Gate counts: {info['gate_counts']}")

# --- Ergebnisse speichern ---
append_hardware_csv({
    'Modell': 'VQC (RealAmplitudes, COBYLA)',
    'Datensatz': ds_label,
    'Backend': backend.name,
    'Optimizer': 'COBYLA',
    'Iterationen': 100,
    'Accuracy': acc, 'F1': f1,
    'Trainingszeit_s': train_time,
    'Inferenzzeit_s': infer_time,
    'Queue_Zeit_s': 'manuell_erfassen',  # s. Hinweis unten
    'Circuit_Depth': info['depth'],
    'Gate_Counts': str(info['gate_counts']),
    'Num_2q_Gates': info['num_2q_gates'],
    'optimization_level': OPTIMIZATION_LEVEL,
    'Job_ID': 'siehe_IBM_Dashboard',
})
```

```
/tmp/ipykernel_1220023/3807324446.py:3: DeprecationWarning: The class ``qisk
it.circuit.library.data_preparation._zz_feature_map.ZZFeatureMap`` is deprec
ated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the zz_feature_
map function as a replacement. Note that this will no longer return a Bluepr
intCircuit, but just a plain QuantumCircuit.
  feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
/tmp/ipykernel_1220023/3807324446.py:4: DeprecationWarning: The class ``qisk
it.circuit.library.n_local.real_amplitudes.RealAmplitudes`` is deprecated as
of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the function qiskit.cir
cuit.library.real_amplitudes instead.
  ansatz      = RealAmplitudes(num_qubits=2, reps=2)
Circuit depth:      36
Gate counts:        {'rz': 31, 'sx': 17, 'cz': 4}
Anzahl Qubits:      156
Circuit size:       52
2-Qubit-Gates:      4

Starte VQC Training (Iris, Hardware)...
/home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages/qiskit_ibm_runt
ime/qiskit_runtime_service.py:1212: UserWarning: This instance has met its u
sage limit. Workloads will not run until time is made available. Check http
s://quantum.cloud.ibm.com/instances/crn%3Av1%3Abluemix%3Apublic%3Aquantum-co
mputing%3Aus-east%3Aa%2F007f7a42e78b474cae2c07ba12745466%3A5b3a3dfa-cdd4-4d9
9-bbcb-5b476b6e13b0%3A%3A for more details.
  warnings.warn(
```

Queue-Zeit und Job-ID manuell nachtragen

VQC setzt intern viele einzelne Jobs ab - eine automatische Queue-Zeit-Messung ist daher nicht trivial. Empfehlung:

1. Gesamtlaufzeit der Zelle notieren
2. Trainingszeit abziehen → grobe Queue-Approximation
3. Job-IDs im IBM Quantum Dashboard unter "Jobs" nachsehen

Die Queue-Zeit ist vor allem für die **Diskussion in der Studienarbeit** relevant (praktische NISQ-Einschränkung: Wartezeit als Teil der Gesamtlaufzeit).

In [ ]:
```python
# Manuelle Nacherfassung - nach dem Lauf ausfüllen
IRIS_QUEUE_ZEIT_S = None   # z.B. 42.0
IRIS_JOB_ID       = None   # z.B. 'd7nsan497osc73dsfbr0'

if IRIS_QUEUE_ZEIT_S and IRIS_JOB_ID:
    print(f"Job ID:     {IRIS_JOB_ID}")
    print(f"Queue-Zeit: {IRIS_QUEUE_ZEIT_S}s")
```

Lauf 2 - VQC Breast Cancer auf IBM Hardware

Konfiguration: COBYLA, 50 Iterationen, optimization_level=1

In [ ]:

```
X_train, X_test, y_train, y_test, ds_label, f1_avg, bc = prepare_data('bc')

feature_map = zz_feature_map(feature_dimension=2, reps=1)
ansatz      = real_amplitudes(num_qubits=2, reps=2)
sampler     = Sampler(backend)

vqc_bc = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=COBYLA(maxiter=50),
    sampler=sampler,
    pass_manager=pm,
)

# --- Circuit-Eigenschaften ---
info = circuit_info(feature_map, ansatz, pm)
print(f"Circuit depth:      {info['depth']}")
print(f"Gate counts:        {info['gate_counts']}")
print(f"Anzahl Qubits:      {info['num_qubits']}")
print(f"Circuit size:       {info['size']}")
print(f"2-Qubit-Gates:      {info['num_2q_gates']}")

print("\nStarte VQC Training (BC, Hardware)...")
t_train_start = time.time()
vqc_bc.fit(X_train, y_train)
train_time = round(time.time() - t_train_start, 4)

t0 = time.time()
y_pred = vqc_bc.predict(X_test)
infer_time = round(time.time() - t0, 4)

acc = accuracy_score(y_test, y_pred)
f1  = f1_score(y_test, y_pred, average=f1_avg)

print(f"\nAccuracy: {acc:.4f}  |  F1: {f1:.4f}")
print(f"Training: {train_time}s  |  Inferenz: {infer_time}s")
print(classification_report(y_test, y_pred, target_names=bc.target_names))

print(f"Backend: {backend.name}")
print(f"optimization_level: {OPTIMIZATION_LEVEL}")
print(f"Circuit depth: {info['depth']}")
print(f"Gate counts: {info['gate_counts']}")

append_hardware_csv({
    'Modell': 'VQC (RealAmplitudes, COBYLA)',
    'Datensatz': ds_label,
    'Backend': backend.name,
    'Optimizer': 'COBYLA',
    'Iterationen': 50,
    'Accuracy': acc, 'F1': f1,
    'Trainingszeit_s': train_time,
    'Inferenzzeit_s': infer_time,
    'Queue_Zeit_s': 'manuell_erfassen',  # s. Hinweis unten
    'Circuit_Depth': info['depth'],
    'Gate_Counts': str(info['gate_counts']),
```

```
    'Num_2q_Gates': info['num_2q_gates'],
    'optimization_level': OPTIMIZATION_LEVEL,
    'Job_ID': 'siehe_IBM_Dashboard',
})
```

In [ ]:
```
# Manuelle Nacherfassung
BC_QUEUE_ZEIT_S = None
BC_JOB_ID       = None

if BC_QUEUE_ZEIT_S and BC_JOB_ID:
    print(f"Job ID:     {BC_JOB_ID}")
    print(f"Queue-Zeit: {BC_QUEUE_ZEIT_S}s")
```

Zusammenfassung - Hardware-Ergebnisse

In [ ]:
```
import pandas as pd

df_hw = pd.read_csv(HARDWARE_CSV)
print("Hardware-Ergebnisse:")
print(df_hw[['Modell','Datensatz','Accuracy','F1',
             'Trainingszeit_s','Circuit_Depth','optimization_level']].to_str

# Vergleich mit Simulator-Baseline
df_all = pd.read_csv(ERGEBNISSE_CSV)
cobyla_sim = df_all[
    df_all['Modell'].str.contains('COBYLA') &
    df_all['Backend'].str.contains('AerSimulator')
]
if not cobyla_sim.empty:
    print("\nSimulator-Baseline (COBYLA):")
    print(cobyla_sim[['Modell','Datensatz','Accuracy','F1','Trainingszeit_s'
```